Homework 4

Giada Fiacchi, Aleksandra Szczurek, Lucia Santini

2025-11-13

1) We want to estimate the probability P (X > 2) where X follows a standard Cauchy (0, 1) distribution with density

$$f(x) = \frac{1}{\pi(1+x^2)} \quad x \in \mathbb{R}$$

So, we need to calculate

$$\int_2^\infty f(x)dx = \int I_A(x)f(x)dx$$

where IA(x) = 1 if x > 2.

a. Compute the probability using the classical Monte Carlo integration and compare it with the true value:

```
#the first approach that we can use to compute the true value of P(X > 2) is
#to compute the difference between 1 and the cumulative distribution function
#from 2 to infinity
true_prob <- 1 - pcauchy(2, location = 0, scale = 1)
true_prob
```

```
## [1] 0.1475836
```

```
#True probability (analytical)
true_p <- 0.5 - atan(2) / pi
true_p
```

```
## [1] 0.1475836
```

That's the analytical method. When we have an integral that provides us a closed solution, it's better to solve the integral manually in place of using the R commands like follows.

```
#True probability - integrate
f <- function(x) 1/(pi * (1 + x^2))
p <- integrate(f, lower = 2, upper = Inf)
p
```

```
## 0.1475836 with absolute error < 1.3e-10
```

```r
#Monte Carlo Integration
n <- 10^5  #we need a large n to guarantee efficiency

set.seed(10)

samples <- rcauchy(n, location = 0, scale = 1)
```

We need to generate n random variates from the standard Cauchy(0, 1) distribution

```r
#we apply the indicator function I_A(x) = 1 if x > 2, 0 otherwise
indicator_vals <- samples > 2
```

The Monte Carlo method uses random samples to approximate the integral like an empirical mean. We need to construct a vector of 0 and 1 that has values =1 when X>2.

```r
#Monte Carlo estimate of P(X > 2)
mc_estimate <- mean(indicator_vals)
mc_estimate
```

```
## [1] 0.14795
```

By this way we use the vector of 0 and 1 and we compute the empirical average value.

```r
#table for comparison
tab<- data.frame(
  Method = c("True analythical value", "Monte Carlo Estimate"),
  Probability = c(true_p, mc_estimate)
)
tab
```

```
##                   Method Probability
## 1 True analythical value   0.1475836
## 2   Monte Carlo Estimate   0.1479500
```

b. Monitor the convergence to the true value with the 95% confidence bounds (with an appropriate graph);

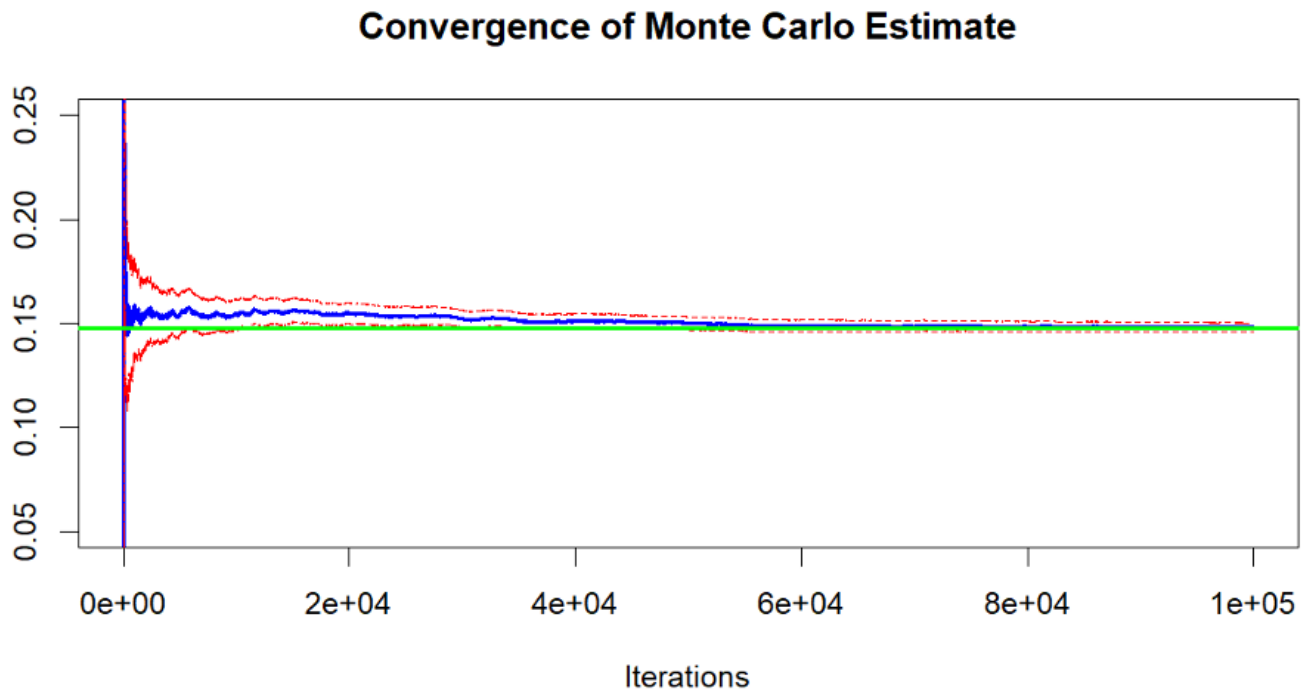
```r
#cumulative Monte Carlo estimates
p_hatt <- cumsum(indicator_vals) / (1:n)

#estimated standard error and 95% CI bounds
se_hat <- sqrt(cumsum((indicator_vals-p_hatt)^2)) / (1:n)
upper <- p_hatt + 1.96 * se_hat
lower <- p_hatt - 1.96 * se_hat

#plot to check the convergence
plot(p_hatt, type = "l", col = "blue", lwd = 2,
```

```
     ylim = c(0.05,0.25),
     xlab = "Iterations",
     ylab = "",
     main = "Convergence of Monte Carlo Estimate")

lines(upper, col = "red", lty = 2)
lines(lower, col = "red", lty = 2)
abline(h = true_p, col = "green", lwd = 2)
```



c.Use the importance sampling with importance density #h(x) = 2/x^2 , x>2 #to compute the probability. Use the inverse transformation method to simulate from h(x) where the cdf of h(x) (with x>2) is

$$h(x) = \frac{2}{x^2}, \quad x > 2$$

FX(x) = integral from 2 to x of f(t)dt = −2/t computing in 2 and x = - 2/x + 1

$$F_X(x) = \int_2^x = f(t)dt = -\frac{2}{t}\Big|_2^x = -\frac{2}{x} + 1$$

and compare MC error and convergence with the previous results #(use par(mfrow=c(1,2)) to draw both the convergence graphs in the same plot).

```r
#we already have the function f so we need just to compute the function h
h <- function(x) {
  ifelse(x>2,(2/x^2),0)
}

#to use the inverse transformation method we need to compute the inverse of the
cumulative distribution function of h(x)
u <- runif(n)
h_inv <- 2 / (1 - u)  #inverse function
x1 <- h_inv

# Define target f(x) and proposal h(x)
g <- function (x) x > 2
f <- function(x) 1 / (pi * (1 + x^2))
h <- function(x) 2 / (x^2)

# weights: w = f(x)/h(x)
w <- function(x) f(x)/h(x)

#IS estimates
IS.MC <- mean(g(x1)* w(x1))
IS.MC
```

```
## [1] 0.1475465
```

```r
IS.SE  <- sqrt ((var(g(x1)* w(x1))) /n )
IS.SE
```

```
## [1] 3.094193e-05
```

```r
#table for comparison
tab<- data.frame(
  Method = c("True analythical value", "Monte Carlo Estimate","Importance
Sampling"),
  Probability = c(true_p, mc_estimate, IS.MC)
)
tab
```

```
##                   Method Probability
## 1 True analythical value   0.1475836
## 2   Monte Carlo Estimate   0.1479500
## 3    Importance Sampling   0.1475465
```

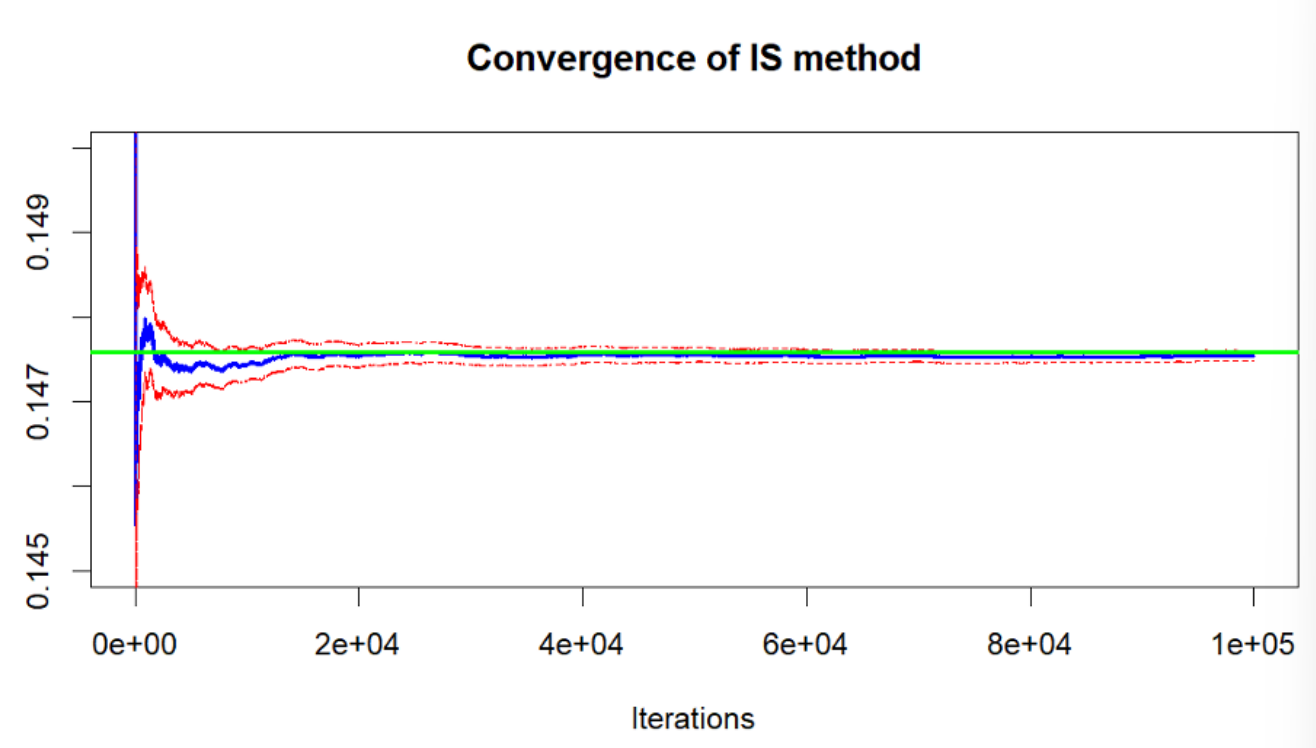
```r
x2 <- g(x1)* w(x1)
estint_IS <- cumsum(x2) / (1:n)

#estimated standard error and 95% CI bounds
se_hat_IS <- sqrt(cumsum((x2 - estint_IS)^2)) / (1:n)
upper_IS <- estint_IS + 1.96 * se_hat_IS
lower_IS <- estint_IS - 1.96 * se_hat_IS

#chek the convergence
```

```r
plot(estint_IS, type = "l", col = "blue", lwd = 2,
     ylim = c(0.145,0.15),
     xlab = "Iterations",
     ylab = "",
     main = "Convergence of IS method")

lines(upper_IS, col = "red", lty = 2)
lines(lower_IS, col = "red", lty = 2)
abline(h = true_p, col = "green", lwd = 2)
```




```r
#comparison of the convergence graphs
par(mfrow=c(1,2))

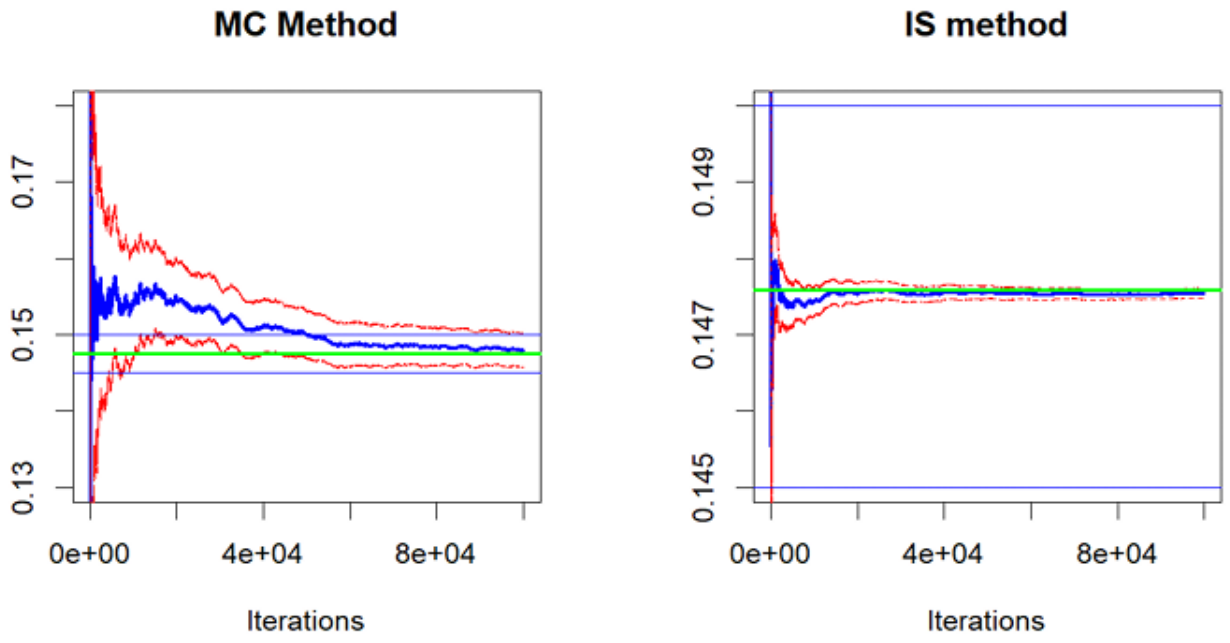
#Monte carlo method
plot(p_hatt, type = "l", col = "blue", lwd = 2,
     ylim = c(0.13,0.18),
     xlab = "Iterations",
     ylab = "",
     main = "MC Method")

lines(upper, col = "red", lty = 2)
lines(lower, col = "red", lty = 2)
abline(h = 0.145 , col = "blue", lwd = 1)
abline(h = 0.15 , col = "blue", lwd = 1)
abline(h = true_p, col = "green", lwd = 2)

#Importance sampling method
plot(estint_IS, type = "l", col = "blue", lwd = 2,
```

```
     ylim = c(0.145,0.15),
     xlab = "Iterations",
     ylab = "",
     main = "IS method")

lines(upper_IS, col = "red", lty = 2)
lines(lower_IS, col = "red", lty = 2)
abline(h = 0.145 , col = "blue", lwd = 1)
abline(h = 0.15 , col = "blue", lwd = 1)
abline(h = true_p, col = "green", lwd = 2)
```



The Monte Carlo estimate shows higher variance and slower convergence, with wider confidence bounds that narrow gradually. In contrast, the Importance Sampling method converges much faster and exhibits significantly lower variance, demonstrating improved estimation efficiency.

2. Consider the computation of the expectation Ef [g(X)] when f is the standard normal N(0, 1) pdf and g(x) = exp(−(x − 3)2/2) + exp(−(x − 6)2/2):

$$E_f[g(X)] = \int_{-\infty}^{\infty} g(x)f(x)$$

This integral may be computed in closed form, and its value is around 0.0746.

a. Compute the integral with the deterministic numerical integration by means of function integrate;

b. Construct a standard Monte Carlo approximation based on a normal N(0, 1) sample of size n = 104 and produce an error evaluation by means of the MC standard error and 95% confidence bounds;

c. Compare the above result with an importance sampling approximation based on an importance function h corresponding to the Unif(−8, −1) distribution and a sample of size n = 104;

d. Compare the standard MC integration with an importance sampling based on importance density h corresponding to the Cauchy distribution with location parameters equal to l = 1.5 and scale parameter equal to s = 1. (The Cauchy distribution with location l and scale s has density;

$$f(x) = \frac{1}{\pi s\left[1 + \left(\frac{x-l}{s}\right)^2\right]}, \quad \forall x.$$

In R the density and the random generation for the Cauchy distribution may be computed by using function dcauchy and rcauchy, respectively). Compute the variance and the relative efficiency of both estimators (classical MC and Importance sampling); provide 95% bounds and draw the corresponding convergence graphs to perform comparison.

```r
#Define g(x) and f(x)
g<-function(x) exp(-(x-3)^2/2)+exp(-(x-6)^2/2)
f<- function (x) dnorm(x, 0,1)

#Define the integrand
gf<-function (x) g(x)*f(x)



#a
detvalue<-integrate(gf, -Inf, +Inf)$value
detvalue
```

```
## [1] 0.07461577
```

```r
#b
set.seed(12)
xb <- rnorm(10^4,0,1)

hat.theta <- mean(g(xb)) #MC estimator
hat.theta
```

```
## [1] 0.07483244
```

```r
MC.se <- sqrt(var(g(xb))/10^4) #MC standard error
MC.se
```

```
## [1] 0.0015225
```

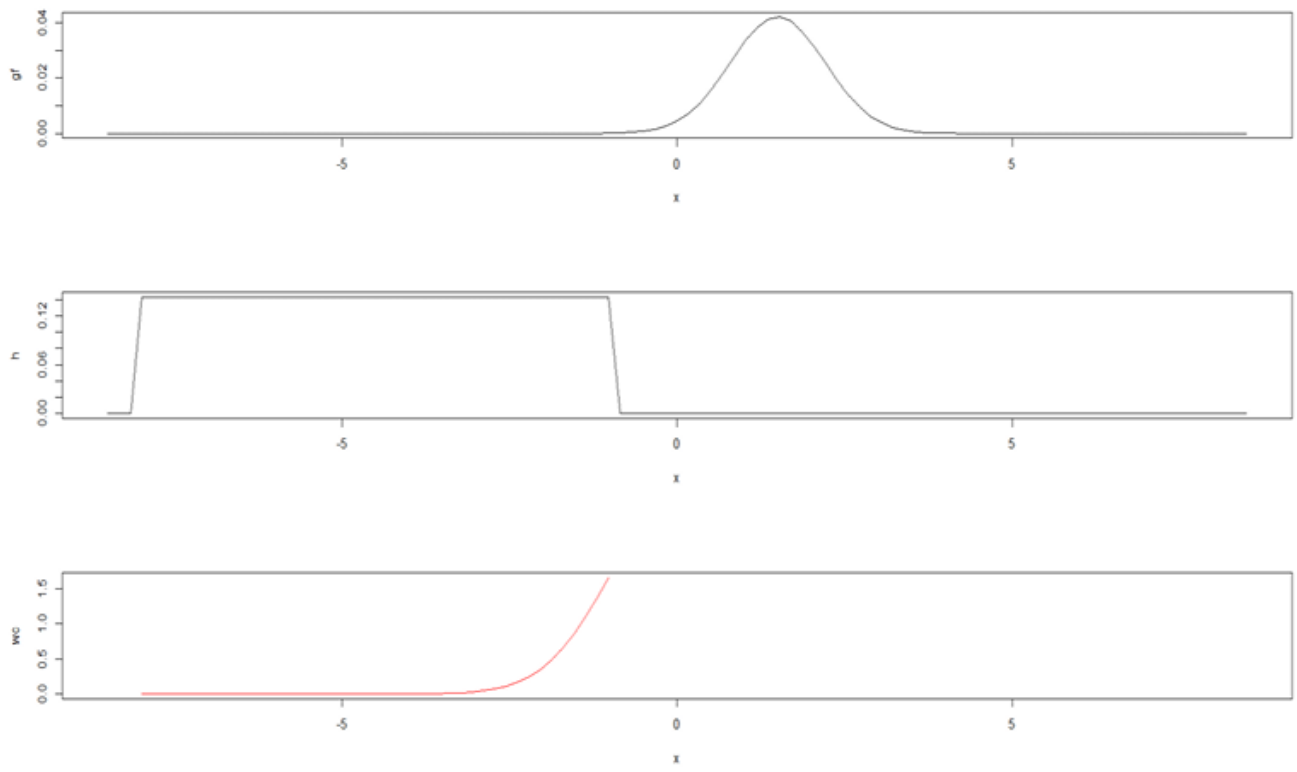
```r
bounds <-c(hat.theta-1.96*MC.se,hat.theta+1.96*MC.se)
bounds #95% confidence interval

## [1] 0.07184834 0.07781654

#c
a<- -8
b<- -1
h <- function(x) {
  val <- 1 / (b - a)
  ifelse(x >= a & x <= b, val, 0)
}

wc<- function(x) f(x)/h(x) #weight function f(x)/h(x)

par(mfrow=c(3,1))
plot(gf, -8.5, 8.5)
plot(h, -8.5, 8.5)
plot(wc, -8.5, 8.5, col="red")
```



The Importance function h(x) samples where x is in [-8, -1]. The Importance function h(x) samples in a region where g(x)*f(x) is nearly zero.

```r
set.seed(12)
x1<-runif(10^4, a, b)
```

```r
#IS estimator
hat.thetais<-mean(g(x1)*wc(x1)); hat.thetais
```

```
## [1] 1.614371e-05
```

```r
#standard error for IS
is.se<-sqrt(var(g(x1)*wc(x1))/10^4);is.se
```

```
## [1] 6.726047e-07
```

```r
#Bounds (95% confidence interval) for IS
boundsis <-c(hat.thetais-1.96*is.se,hat.thetais+1.96*is.se)
boundsis
```

```
## [1] 1.482540e-05 1.746201e-05
```

```r
#Table for comparison
tab <-data.frame(
  "-"=c("Approximation", "Standard Error", "Lower Bound", "Upper Bound"),
  "Deterministic value" = c(round(detvalue,8), "-", "-", "-"),
  "Monte Carlo" = c(hat.theta, MC.se, bounds[1], bounds[2]),
  "Importance Sampling" = c(hat.thetais, is.se, boundsis[1], boundsis[2])
)
tab
```

```
##               X. Deterministic.value Monte.Carlo Importance.Sampling
## 1  Approximation          0.07461577  0.07483244        1.614371e-05
## 2 Standard Error                   -  0.00152250        6.726047e-07
## 3    Lower Bound                   -  0.07184834        1.482540e-05
## 4    Upper Bound                   -  0.07781654        1.746201e-05
```

The MC approximation is very close to the deterministic value. The 95% confidence interval contains the true value. The IS approximation is extremely far from the correct value. This indicates a complete failure of the Importance Sampling method with the uniform density U(-8, -1). The 95% confidence interval is tight but far to the true value, confirming that the estimator is both highly precise (low standard error) and highly inaccurate. The failure is due to a poor choice of the importance density h because the function g(x)*f(x) lies primarily in the positive tail of the normal distribution, but h only cover the negative tail.

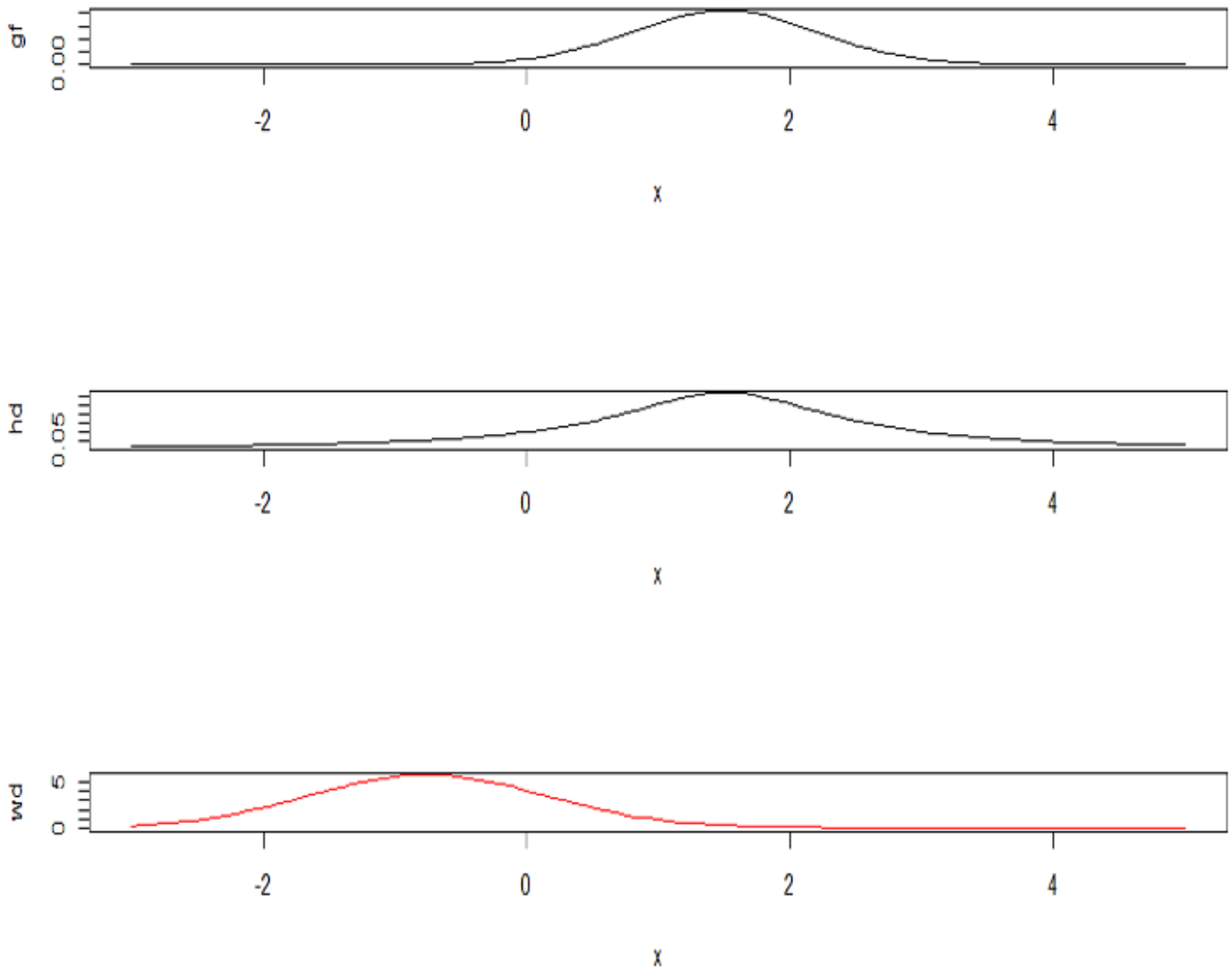
```r
#d
l<- 1.5
s<- 1

#cauchy density
hd <- function(x) 1/(pi*s*(1+((x-l)/s)^2))

#weight function
wd<- function(x) f(x)/hd(x)

par(mfrow=c(3,1))
plot(gf, -3, 5)
```

```
plot(hd, from=-3, to=5)
plot(wd, -3, 5, col="red")
```



There is excellent overlap between g(x)f(x) and the importance function h(x). The weight function is relatively flat and this is a good predictor of low variance for the IS estimator.

```
set.seed(12)
xd<-rcauchy(10^4, l, s)

#IS estimator
hat.thetad<-mean(g(xd)*wd(xd)); hat.thetad

## [1] 0.07486254

#Standard error
d.se <- sqrt(var(g(xd)*wd(xd))/10^4)
d.se

## [1] 0.0005619149
```

```r
#Bounds
boundsd <-c(hat.thetad-1.96*d.se,hat.thetad+1.96*d.se)
boundsd
```

```
## [1] 0.07376118 0.07596389
```

```r
#variance of Standard MC integration
MC.se^2
```

```
## [1] 2.318007e-06
```

```r
#variance of the importance sampling based on importance density corresponding to
the Cauchy distribution
d.se^2
```

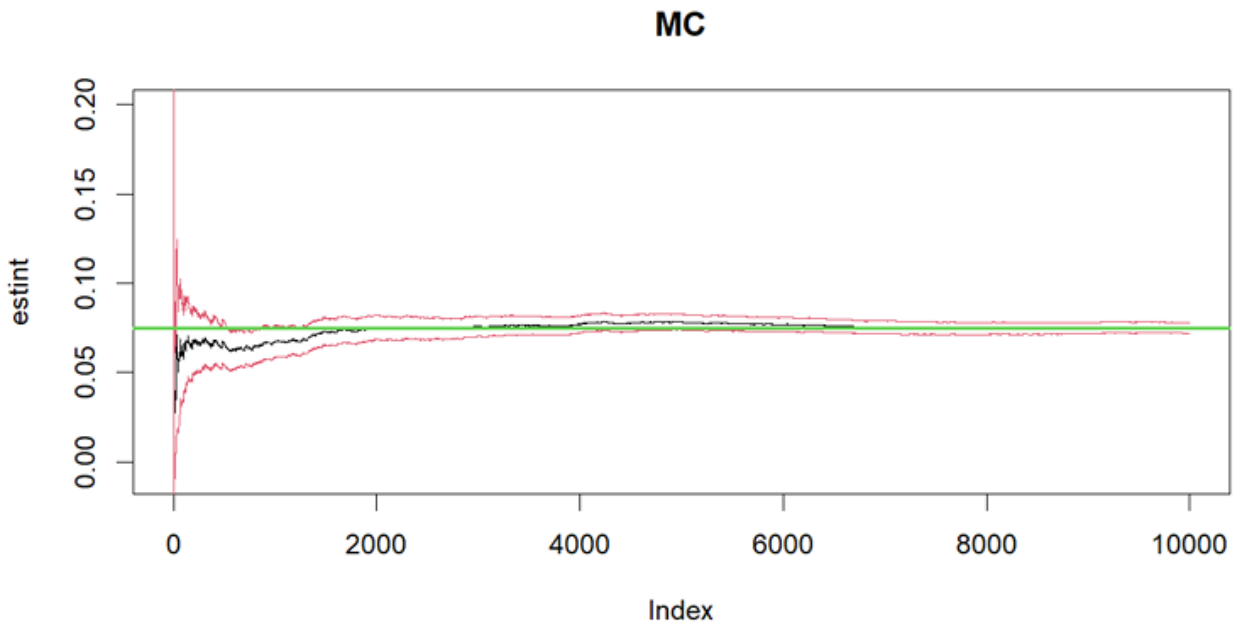
```
## [1] 3.157484e-07
```

```r
#compute the relative efficiency
(MC.se^2)/(d.se^2)
```

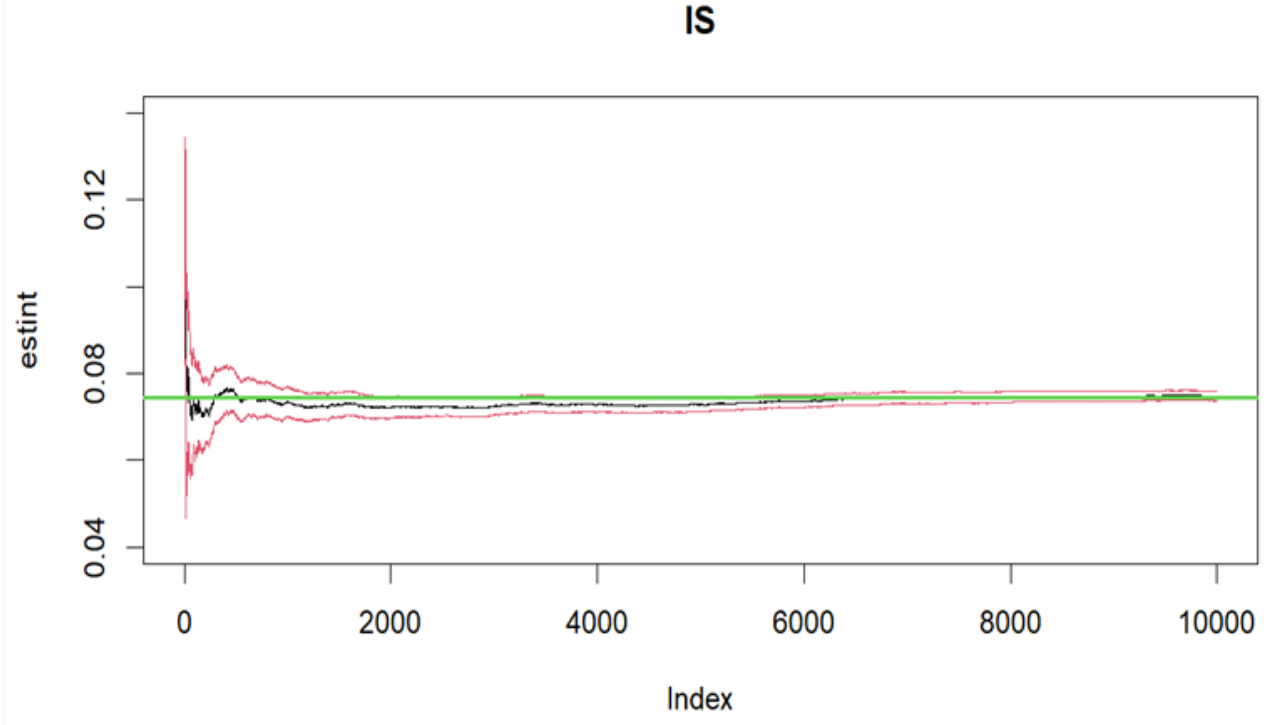
```
## [1] 7.341312
```

The relative efficiency is 7.341312, then RE>1: the importance sampling based on the Cauchy density is more efficient than the standard Monte Carlo integration.

```r
#Standard MC integration convergence
par(mfrow=c(1,1))
xx1 <- g(xb)
estint <- cumsum(xx1)/(1:10^4) #cumulative mean
esterr <- sqrt(cumsum((xx1-estint)^2))/(1:10^4)
plot(estint,type="l", ylim=c(-0.01, +0.2), main="MC")
lines(estint-1.96*esterr,col=2)
lines(estint+1.96*esterr,col=2)
abline(h=detvalue, col=3,lwd=2.5) #true deterministic value
```




```r
#importance sampling convergence
xx2 <- g(xd)*wd(xd)
estint <- cumsum(xx2)/(1:10^4)
esterr <- sqrt(cumsum((xx2-estint)^2))/(1:10^4)
plot(estint,type="l", ylim=c(0.04, +0.14), main="IS")
lines(estint-1.96*esterr,col=2)
lines(estint+1.96*esterr,col=2)
abline(h=detvalue, col=3,lwd=2.5)
```



The convergence plots show that the Importance Sampling estimator with the Cauchy density converges faster and with smaller variance than the Standard Monte Carlo estimator. The narrower red confidence bands in the IS plot indicate higher efficiency. Both estimators converge to the true deterministic value, but the Cauchy-based IS method achieves lower variance for the same number of samples.

3. Consider the following integral

$$\int_0^{10} \exp(-2 \mid x-5 \mid)dx$$

a. *Compute the integral using the classical MC integration and provide standard error and 95% bounds*
b. *Use the importance sampling with importance density h(x) ~N(5, 1) to solve the integral and compare MC error with the previous results*

$$(Hint:\ re\text{-}write\ the\ integral\ as\ \int_0^{10} 10\exp(-2 \mid x-5 \mid)f(x)dx\ with\ f(x) \sim U(0,10))$$

POINT A

```r
set.seed(123)


g <- function(x) 10 * (exp(-2 * abs(x - 5)))
f <- function(x) dunif(x,0,10)

g_true <- function(x) (exp(-2 * abs(x - 5)))

#Numerical deterministic integration
det.int <- integrate(function(x) g_true(x), lower=0, upper=10)
det.int

## 0.9999546 with absolute error < 2.8e-14

#Standard Monte-Carlo estimate
n <- 10^5
x <- runif(n,0,10)

MC <-mean(g(x)); MC

## [1] 0.9998279

se.MC <- sqrt(var(g(x))/n); se.MC

## [1] 0.006312751

#Bounds
bounds <- c(MC-1.96*se.MC,MC+1.96*se.MC)
bounds
```

```
## [1] 0.9874549 1.0122009
```

POINT B

```r
# Importance Sampling estimate
g1 <- function(x1) 10 * (exp(-2 * abs(x1 - 5)))
h <- function(x1) dnorm(x1, 5, 1) #importance density
f1 <- function(x1) dunif(x1,0,10)

#define the weight
w <- function(x1) f1(x1)/h(x1)
x1 <- rnorm(n, 5, 1)

#IS estimator
MC.IS <- mean(g1(x1) * w(x1));MC.IS
```

```
## [1] 1.000043
```

```r
se.IS <- sqrt(var(g1(x1)* w(x1)) / n);se.IS
```

```
## [1] 0.001888722
```

```r
#Importance sampling and MC comparison
tab = c(MC,MC.IS)
tab = cbind(theta=tab,SE = c(se.MC, se.IS))
tab=cbind(tab,RelEff=tab[,2]/min(tab[,2]))
rownames(tab) = c("Classical MC", "ImportanceSamp1")
tab
```

```
##                      theta          SE   RelEff
## Classical MC    0.9998279 0.006312751 3.342339
## ImportanceSamp1 1.0000426 0.001888722 1.000000
```

```r
#Efficiency
se.IS^2/se.MC^2
```

```
## [1] 0.08951563
```

We can see that efficiency ratio is less than 1 so it's better the importance sampling method.